

2025 届本科生学士学位论文

学校代码: 10269



华东师范大学
East China Normal University

本科生毕业论文

基于多智能体 LLM 的 Text2SQL 系统设计与实现

Design and Implementation of a Multi-Agent LLM-Based Text2SQL System

姓 名: 谢 骐

学 号: 10215102427

学 院: 计算机科学与技术学院

专 业: 计算机科学与技术

指导教师: 杨 燕

职 称: 副教授

2025 年 月

华东师范大学学位论文诚信承诺

本毕业论文是本人在导师指导下独立完成的，内容真实、可靠。本人在撰写毕业论文过程中不存在请人代写、抄袭或者剽窃他人作品、伪造或者篡改数据以及其他学位论文作假行为。

本人清楚知道学位论文作假行为将会导致行为人受到不授予/撤销学位、开除学籍等处理（处分）决定。本人如果被查证在撰写本毕业论文过程中存在学位论文作假行为，愿意接受学校依法作出的处理（处分）决定。

承诺人签名：

日期： 2025 年 月 日

华东师范大学学位论文使用授权说明

本论文的研究成果归华东师范大学所有，本论文的研究内容不得以其它单位的名义发表。本学位论文作者和指导教师完全了解华东师范大学有关保留、使用学位论文的规定，即：学校有权保留并向国家有关部门或机构送交论文的复印件和电子版，允许论文被查阅和借阅；本人授权华东师范大学可以将论文的全部或部分内容编入有关数据库进行检索、交流，可以采用影印、缩印或其他复制手段保存论文和汇编本学位论文。

保密的毕业论文（设计）在解密后应遵守此规定。

作者签名：

导师签名：

日期： 2025 年 月 日

目录

基于多智能体 LLM 的 Text2SQL 系统设计与实现

摘要:

Text2SQL (Text-to-SQL) 旨在将自然语言查询转译到 SQL 语句, 允许人们用日常用语访问数据库。传统的 Text2SQL 运用基于模板/规则的方法或深度学习, 在应对模式复杂的数据库和多样化查询时表现有限。近几年 LLM 的兴起为 Text2SQL 提供了新范式, 其强大的语言理解和生成能力在 Text2SQL 任务中表现出众^[1]。然而 LLM 不是银弹, 直接调用单一 LLM 会面临巨量的 token 消耗成本、难以避免的数据隐私风险、以及生成结果的不稳定性。

本文在此提出一种多智能体架构赋能的 Text2SQL 系统, 设计多个分工明确的 LLM 协同工作, 以构建端到端的流水线: 从用户意图澄清、知识检索、prompt 生成、SQL 生成、多答案校验、到结果反馈。我们的系统充分利用 LLM 在复杂推理和语言生成上的优势 (注意, SQL 自身的语法规则就是对标自然语言而设计出来的), 应用多智能体协作机制提高查询的准确性和稳定性。我们的优化策略包括: RAG 知识检索, 多智能体投票决策、异步并行处理、上下文缓存、会话剪枝、partial mode 前缀引导、结构化输出约束、以及 RDBMS 事务机制保障数据安全。

实验结果表明, 相较于传统方案, 本系统在查询准确率、执行效率、结果稳定性方面均有显著提升。

关键词: Text2SQL, LLM, 多智能体, 数据库

Design and Implementation of a Multi-Agent LLM-Based Text2SQL System

Abstract:

Text2SQL (Text-to-SQL) aims to translate natural language queries into SQL statements, allowing users to access databases using everyday language. Traditional Text2SQL approaches, which rely on template/rule-based methods or deep learning, struggle with complex database schemas and diverse query patterns. The recent rise of Large Language Models (LLMs) has introduced a new paradigm for Text2SQL^[1], leveraging their powerful language understanding and generation capabilities to achieve remarkable results. However, LLMs are not a silver bullet. Directly using a single LLM poses challenges such as high token consumption costs, inherent data privacy risks, and instability in generated results.

To address these challenges, this paper proposes a multi-agent architecture-powered Text2SQL system, where multiple LLMs collaborate within a well-structured pipeline—from user intent clarification, knowledge retrieval, and prompt generation to SQL generation, multi-answer validation, and result feedback. Our system fully capitalizes on LLMs' strengths in complex reasoning and language generation (notably, SQL syntax itself is designed to align with natural language concepts). By employing a multi-agent collaboration mechanism, we enhance query accuracy and stability. Our optimization strategies include Retrieval-Augmented Generation (RAG) for knowledge retrieval, multi-agent voting mechanisms, asynchronous parallel processing, context caching, session pruning, partial mode prefix guidance, structured output constraints, and RDBMS transaction mechanisms to ensure data security.

Experimental results demonstrate that, compared to traditional approaches, our system significantly improves query accuracy, execution efficiency, and result stability.

Keywords: Text2SQL, LLM, Multi-Agent, Database

1、绪论

1.1 研究背景

将自然语言转换为 SQL 语句始终是数据库领域和 NLP 领域的重要课题。

Text2SQL 技术充当自然语言与数据库之间的桥梁，如此人们即可以日常用语查询数据库，大大降低数据库的使用门槛。

传统上，Text2SQL 多采用基于规则的方法，将用户问题映射至预定义的 SQL 模板。这对简单场景是有效的，但数据库的模式时常很复杂，且自然语言的表述多变，这些场景下传统方法就捉襟见肘，难以扩展。后来，深度学习方法兴起，采用“编码器-解码器”神经网络生成 SQL。不过训练这些神经网络模型往往需要大规模标注的数据集，因此泛化能力有限，对于跨领域的新数据库，性能差强人意。

当下，大规模语言模型（LLM）如 GPT-4、DeepSeek 的出现给 Text2SQL 带来了新的契机。LLM 在自然语言理解和生成上崭露出前所未有的能力，使得 LLM 驱动的 Text2SQL 更好地泛化到不同领域的数据库；得益于其强大的推理能力，甚至在零样本或小样 prompt 下仍能生成复杂的 SQL 查询。

1.2 相关工作

下面按时间线和技术路线对主要方法进行回顾。

1.2.1 基于模板和规则的方法

早期 Text2SQL 系统极度依赖“模板匹配”和“规则解析”。典型做法是预先定义一组自然语言问句到 SQL 模板的映射，程序根据用户查询的关键词或句法结构匹配特定的 SQL 模板，替换槽位以生成 SQL。例如，IBM 早期的系统可能预置规则：“T 中哪些记录的 F 是 V”对应 SQL 结构 `SELECT * FROM T WHERE F = V` 等。^[2]

它的缺点很明显：（1）可扩展性差：难以覆盖丰富的自然语言表达；（2）适应性弱：无法处理未收录的语法或复杂查询。

有研究者开始探索数据驱动的学习方法，这可以减少对人工规则的依赖。例如，交互式 NLIDB（Natural Language Interface to Database）系统就尝试结合用户澄清提问来改进准确率。^[3]

总之，模板和规则方法奠定了 Text2SQL 的初步基础，其虽有局限性但也推动了后续数据驱动方法的发展。

1.2.2 基于深度学习的方法

1.2.2.1 序列到序列模型

引入神经网络使 Text2SQL 进入了数据驱动的新阶段。典型的是将 Text2SQL 视为机器翻译问题，采用“Encoder-Decoder”架构，编码器将自然语言问题编码为向量表示，解码器根据表示生成对应的 SQL。

早期工作中，基于循环神经网络（RNN，尤其是 LSTM）的模型取得了一定成功。例如，Seq2SQL 模型通过强化学习优化生成的 SQL^[4]；SQLNet 引入语法模板约束以避免部分错误；TypeSQL 利用问题中的实体类型信息辅助生成；优语义解析方法 SyntaxSQLNet 更是直接解析 AST 以生成 SQL。

但 LSTM 模型仍有缺点，“长期依赖不足”和“跨域泛化弱”使它们难以有效处理问题中的长距离依赖。而且这些模型通常需要在大规模标注的语料上进行训练，可以猜到对于新领域（未知数据库模式）往往表现不佳，事实也确实如此。

1.2.2.2 基于 Transformer 和预训练的模型

Transformer 通过“自注意力机制”能够更有效地捕捉长程依赖和复杂结构关系。

如基于 Transformer 的 SQLova、GPSQL 等模型在 Spider 等 baseline 上拥有领先性能。GraPPa 通过在大规模表语料上预训练增强模型对数据库模式的表示能力；RAT-SQL 利用关系自注意力网络结合数据库模式信息，实现了对模式的有效编码，被认为是 Spider Challenge 中的标杆模型之一。^[5]

这些预训练模型显著提升了跨领域场景下的性能，缩小与人工编写 SQL 之间的差距。但深度学习模型普遍缺乏对错误的纠正能力，往往“一次生成定成败”，缺少交互式改正的机制。

1.2.3 基于大语言模型（LLM）的方法

大语言模型（如 GPT-3 系列）的出现，为 Text2SQL 提供了新的范式，即“基于 prompt 的零样本/小样本学习”。只需精心设计 prompt，提供一些示例，LLM 就能理解任务要求并生成相应的 SQL。

有研究者探索了各种 Prompt Engineering 技巧，如链式思维提示（Chain-of-Thought）引导 LLM 逐步推理复杂查询，将问题分解为子问题并逐一求解，再合成最终 SQL。^[6] Wang et al. (2022) 提出的“自一致性（Self-Consistency）”解码策略进一步提升了链式推理的可靠性。^[7]

另一些工作尝试对开源 LLM 进行专门的 Text2SQL 微调，以在保留 LLM 强大能力的同时，注入领域知识和术语。微调的好处在于模型可以学到更贴合数据库查询的风格和约束，从而减少语法错误和不相关输出。但微调需要大规模高质量的训练数据，如果数据库领域与训练语料差异很大，很可能出现知识不充分的问题。

1.3 本文贡献

将为应对上述挑战，本文设计并实现了一个基于多智能体 LLM 的 Text2SQL 系统。

我们的主要贡献包括：（1）多智能体协作架构：提出了由多个 LLM 智能体组成的流水线式解决方案。各智能体承担不同角色，包括：用户意图澄清、prompt 构建、SQL 生成、执行校验、结果反馈等，实现模块化分解和协同处理。该架构结合 LLM 强大的语言能力和多智能体的灵活性，使系统具备较强的适应性。（2）检索增强的 prompt 生成：引入 RAG 机制，在 SQL 生成前自动检索相关的文档和示例作为辅助知识。这能有效缓解 LLM 的幻觉问题，生成有据可依的 SQL，有望提高在特定领域和边缘案例下的准确性。（3）结果校验与自我修正：设计 SQL 执行校验和错误反馈环节，在真实数据库上执行多轮来验证生成 SQL 的正确性，后将错误信息反馈给生成智能体进行自我修正。已有研究证明类似的多智能体校对机制可提升查询准确率。^{[8][9]} 我们的方法实现了自动错误检测和迭代改进，免去人工介入。

（4）全面的优化策略与工程实现：针对 LLM 的性能和确定性问题，我们提出了多项成效可观的优化策略，如多智能体投票融合提高答案准确度、异步并行的 HTTP 调用加速响应、上下文缓存和长对话剪枝减少冗余计算、partial mode 前缀约束输出、JSON 格式结构化输出提高可解析度、以及通过数据库事务保证执行安全等。由此系统的实用性和健壮性得到保障。（5）综合评估：我们有丰富的实验，从准确率、执行效率、稳定性等多方面评估系统，并与传统单智能体方案进行比较。本系统在 Spider 等 baseline 数据集上的查询准确率远超单智能体 LLM baseline，在真实业务数据库上成功应对了一系列模糊查询场景，多智能体方案的有效性从而得到验证。我们还分析了当前系统的局限，指明未来的改进方向。

当下，大规模语言模型（LLM）如 GPT-4、DeepSeek 的出现给 Text2SQL 带来了新的契机。LLM 在自然语言理解和生成上崭露出前所未有的能力，使得 LLM 驱动的 Text2SQL 更好地泛化到不同领域的数据库；得益于其强大的推理能力，甚至在零样本或小样 prompt 下仍能生成复杂的 SQL 查询。

2、基础概念

介绍系统细节前，本章先阐述与本研究相关的几个基础概念。包括注意力机制、大语言模型（LLM）、检索增强生成（RAG），平均语义文本检索（TF-IDF + cosine）。

2.1 注意力机制

注意力机制（Attention Mechanism）最初由 Bahdanau *et al.* (2014) 提出，以改进机器翻译中 RNN “编码器-解码器”对长序列的处理。^[10] 这个理论模拟了人类注意力的选择性：面对冗长的信息，人类会选择性地关注与当前任务相关的要点。注意力机制解决了传统编码器将整个输入压缩成单一向量的不充分问题，使得长句子的关键信息不至于淹没在整体表示中。

现代的大语言模型（LLM）多是基于 Transformer 架构，靠多层自注意力建模语言。注意力机制赋予了 LLM 强大的并行建模长文本的能力，使其能够理解复杂的查询语句和数据库模式之间的匹配关系，这也是 LLM 相比早期模型在 Text2SQL 任务上效果突出的原因之一。

2.2 大语言模型（LLM）

“大语言模型”指参数规模巨大的深度神经网络模型，通常基于 Transformer 架构，在海量文本语料上自监督预训练，对自然语言进行深度的理解和生成。LLM 的“Few-Shot Learning”能力极为强大：即使不专门针对某任务训练，通过 prompt 提供少量示例，LLM 也能在新任务上产生相当好的结果。^[11] 对于 Text2SQL 任务，LLM 能够“理解”自然语言问题并结合给定的数据库 schema 直接生成 SQL 查询。LLM 还掌握了丰富的语言模式和一定的逻辑推理能力，因而表现出更强的泛化性。

纵使 LLM 存储了海量知识，它的训练数据是静态的且覆盖有限，因此 LLM 对特定领域中实时更新的信息可能欠缺了解。尤其在数据库查询场景中，LLM 本身并不“知道”用户数据库的具体内容，因此需要设法将数据库 schema 等上下文提供给它。LLM 还存在“幻觉（hallucination）”，偶尔会编造看似合理但实则错误的答案。^[12] 我们的系统结合 RAG 等手段，努力让 LLM “知有所依”，在生成 SQL 时参考真实数据库文档或示例，减少无根据的胡乱猜测。

成本是另一个无法忽视的难题：LLM 的推理往往需要消耗数百至上千 tokens，且耗时极久。本系统通过优化对话流程和并行机制尽量抬高容错率以使廉价 LLM（意味着更易出错）成为可能，并减少生成耗时。

2.3 检索增强生成 (RAG)

检索增强生成 (Retrieval-Augmented Generation, RAG) 是一种将生成式模型与外部知识库相结合的技术。^[13] RAG 通常包含两部分：检索器和生成器。检索器根据用户问题从知识库（可以是文档集合、数据库、已知问答对等）中找到若干条相关内容；然后将这些外部知识与原问题一起送入生成模型，综合生成最终回答。RAG 的优势在于“将封闭的语言模型变成开放的问答系统”，利用外部最新、权威的数据来提高准确性。

对于 Text2SQL 来说，RAG 可以体现在：在生成 SQL 之前，先从 SQL 文档库或示例库中检索与当前查询相关的语法示例或业务知识，作为 prompt 的一部分提供给 LLM，以帮助其更准确地构造 SQL。更重要的是，RAG 还有助于降低模型对上下文长度的需求：与其把整份文档说明都塞给 LLM，不如先检索出相关部分再提供，减少噪声干扰、降低 token 消耗。

2.3.1 算法分析

我们的系统实现了一个轻量级 RAG 模块：维护一个包含常用 SQL 查询示例和数据库文档的向量索引，当接到用户查询时，检索出最相关的若干条内容，插入 prompt 上下文。

有以代码中的 `RAG.retrieve_context(self, query: str, top_k: int) -> list[str]` 为例，该方法基于用户查询 (query) 从文档 (sql_doc.txt) 中获取最相关的 top_k 条信息。

首先读取文档 `sql_doc.txt`，按空行切分，得到 n 个片段：

$$\mathcal{C} = \{C_1, C_2, \dots, C_n\}$$

我们采用了 `sentence-transformers/all-MiniLM-L6-v2` 预训练模型计算每个片段 C_i 的嵌入向量 E_{C_i} ：

$$H = \text{Transformer}(C_i) \in \mathbb{R}^{L \times d}$$

其中： H 是隐藏状态矩阵， L 是序列长度， d 是嵌入维度。使用**加权平均池化 (Mean Pooling)** 计算最终的片段 embedding：

$$E_{C_i} = \frac{\sum_{j=1}^L H_j \cdot M_j}{\sum_{j=1}^L M_j}$$

其中 M_j 是 `attention_mask`，确保填充部分不影响计算。最后归一化以提高检索稳定性：

$$E_{C_i, \text{norm}} = \frac{E_{C_i}}{\|E_{C_i}\|}$$

计算完片段的 embedding 后，构建 FAISS 近邻搜索索引。FAISS (IndexFlatL2) 以 L2 距离为准则，存储 n 个片段嵌入：

$$\mathcal{I} = \text{faiss.IndexFlatL2}(d)$$

将所有 E_{C_i} 添加至索引：

$$\mathcal{I}.add(\{E_{C_1}, E_{C_2}, \dots, E_{C_n}\})$$

对用户查询 query 计算嵌入 E_q ：

$$E_q = \frac{\sum_{j=1}^L H_j \cdot M_j}{\sum_{j=1}^L M_j}$$

并归一化：

$$E_{q, \text{norm}} = \frac{E_q}{\|E_q\|}$$

最后进行近邻搜索，计算查询嵌入 E_q 与文档嵌入 E_{C_i} 之间的欧几里得距离：

$$D(E_q, E_{C_i}) = \sum_{j=1}^d (E_{q,j} - E_{C_i,j})^2$$

前 top_k 个最小距离的片段索引即为：

$$\mathcal{R} = \underset{\{C_i\}}{\operatorname{argmin}} D(E_q, E_{C_i})$$

对应片段 $\{C_{r_1}, C_{r_2}, \dots, C_{r_k}\}$ ，此即最终的返回值。

2.4 平均语义文本检索

TF-IDF 是一种统计方法，可以评估一份语料库中的其中一段文本的重要程度。通过 TF-IDF 算法将文本转换为向量表示，即可利用 cosine 计算出文本之间的相似度。

我们的系统结合这两种算法，构造出相似度矩阵。通过遍历该矩阵，即可在若干文本片段中，找出与平均语义距离最小的文本。可以认为它就是这份语料库中最具代表性的文本。

2.4.1 算法分析

我们以代码中的 `most_representative_of(responses: list[str]) -> str` 为例。它负责从 `responses` 列表中找出**最具代表性**的文本。

首先对 `responses` 进行 TF-IDF 计算。假设 `responses` 共有 $n=\text{len}(\text{responses})$ 段文本，每个文本的词汇空间大小为 m ，则 TF-IDF 矩阵 M 是一个 $n \times m$ 的矩阵：

$$M = [\mathbf{v}_1, \mathbf{v}_2, \dots, \mathbf{v}_n]^T$$

其中， $\mathbf{v}_i$ 是第 i 个文本的 TF-IDF 向量。知道了文本的数学表示，就可以借此计算它们两两之间的余弦相似度（Cosine Similarity）。文本 i 和文本 j 之间的相似度为：

$$\text{sim}(i, j) = \frac{\mathbf{v}_i \cdot \mathbf{v}_j}{\|\mathbf{v}_i\| \|\mathbf{v}_j\|}$$

这样就可以得到 $n \times n$ 的余弦相似度矩阵 S ：

$$S = \begin{bmatrix} 1 & \text{sim}(1,2) & \text{sim}(1,3) & \dots & \text{sim}(1,n) \\ \text{sim}(2,1) & 1 & \text{sim}(2,3) & \dots & \text{sim}(2,n) \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \text{sim}(n,1) & \text{sim}(n,2) & \dots & & 1 \end{bmatrix}$$

其中，对角线上的值都是 1（因为文本和自身的相似度恒为 1）。对于每个文本 i ，它的平均相似度计算公式为：

$$\bar{s}_i = \frac{1}{n-1} \sum_{\substack{j=1 \\ j \neq i}}^n S_{i,j}$$

即，取该文本与所有其它文本的相似度总和，再除以 $n-1$ 进行归一化。选取平均相似度最高的文本 $\text{best_index} = \text{argmax}_i \bar{s}_i$ 作为最具代表性文本返回。

2.4.2 代码示例

```
def most_representative_of(responses: list[str]) -> str:
    # 将文本转为 TF-IDF 向量:
    tfidf_matrix = (
        sklearn.feature_extraction.text.TfidfVectorizer()
        .fit_transform(responses)
    )
    # 计算余弦相似度矩阵:
    sim_matrix = sklearn.metrics.pairwise.cosine_similarity(tfidf_matrix)
    # 计算每个文本与其他文本的平均相似度:
    avg_similarities = []
    for i in range(len(responses)):
        # 去除自身相似度 (即 1.0), 除以其它文本个数:
        avg_similarities.append(
```

```

(numpy.sum(sim_matrix[i]) - 1) / (len(responses) - 1)
if len(responses) > 1
else 1.0
)
# 找出平均相似度最高的文本:
best_index = int(numpy.argmax(avg_similarities))
return responses[best_index]

```

3、概要设计

3.1 业务流程

3.2 功能模块介绍

如图 2-2 系统用例图所示，系统的业务流程分为两部分，购买 WF 和 In 购买。

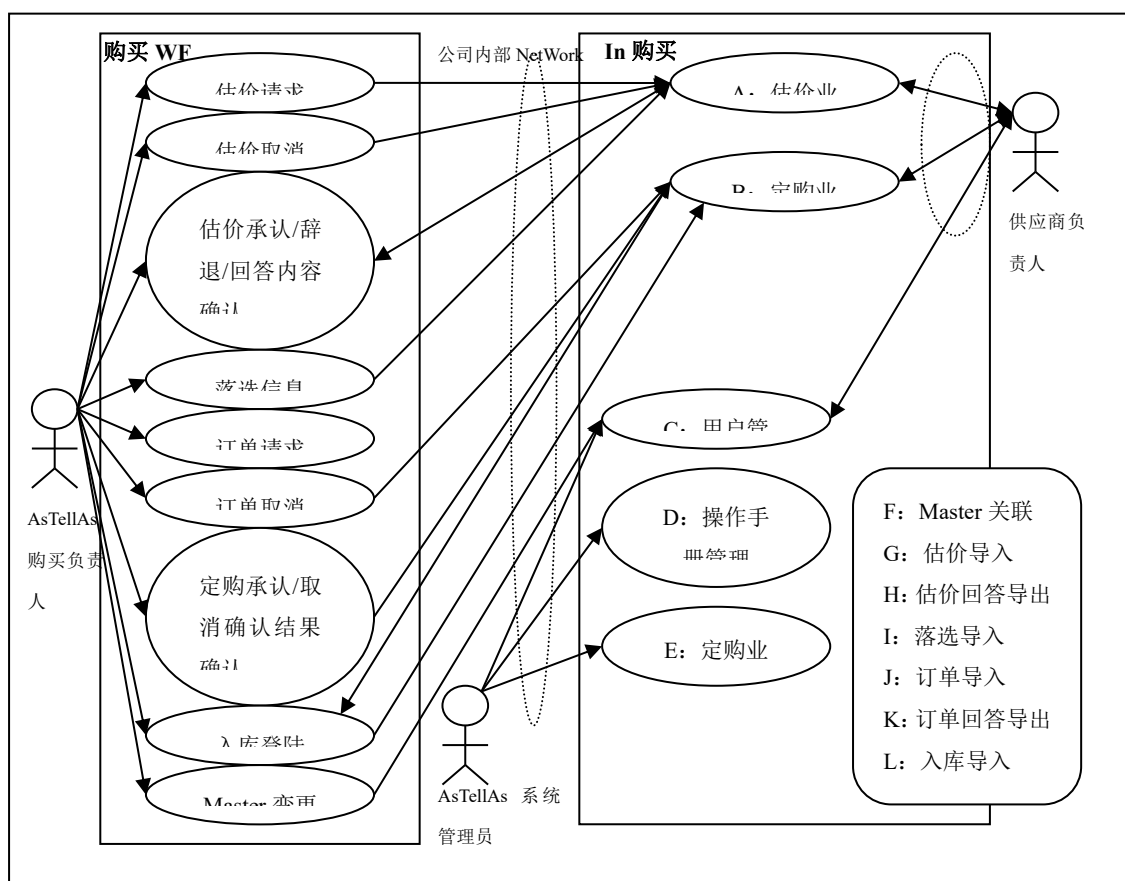


图 2-2 系统用例图

Figure 2-2 System Use Case

4、详细设计

4.1 数据库的设计

4.2 细设计书

4.3 界面设计

5、系统实现

5.1 界面

6、关键算法与数据结构

7、系统测试

7.1 测试计划

7.1.1 单元测试

单元测试主要是对画面显示的测试，主要测试点如表 4-1 所示。

表 4-1 单体测试表

Table 4-1 Unit Test Case

输入系	输入正常值 (最小值 < 值 < 最大值)
	输入系统异常值 (值=最小值-1, 值=最大值+1)
	输入系统正常值 (边际测试) (值=最小值, 值=最大值)
	未输入值
显示系	标题 (Title) 显示
	说明文显示
	显示相应业务的左菜单
	显示 Text 项目 (不换行)
	按需要显示排序按钮
	添加附件参照
操作系	IE 返回按钮按下后画面迁移
	IE×按钮按下后, 系统終了
	下一画面迁移按钮按下后的迁移
	上一画面迁移按钮按下后的迁移

在表格的上方要有表序号和注释，表序号要在正文中被引用

表名放置在表格正上方，中英文对照

表格尽量放在一页内，不要跨页，居中放置

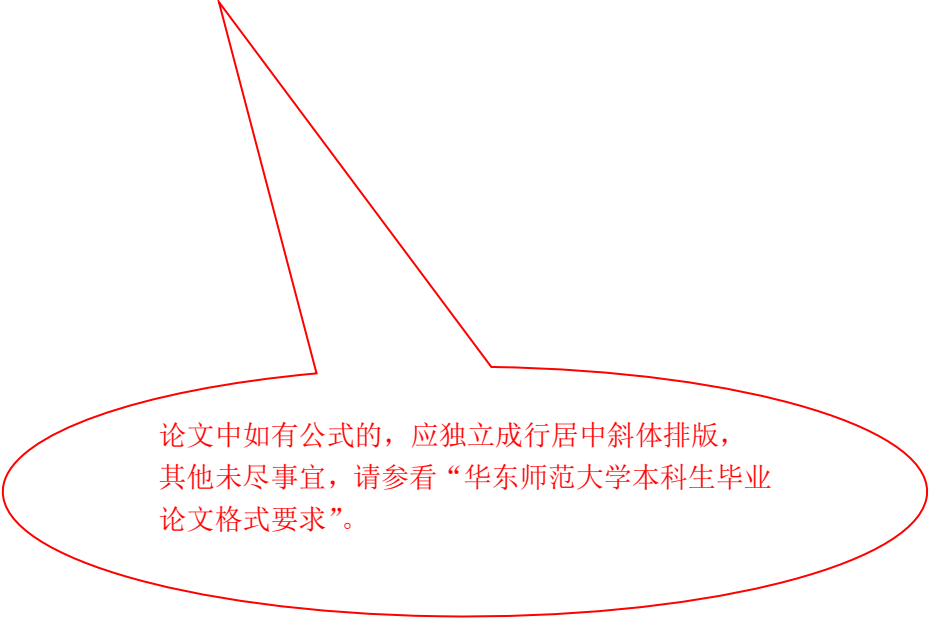
7.1.2 结合测试

7.1.3 模拟测试

7.1.4 随机测试

8、测试用例与测试报告

9、总结和展望



论文中如有公式的，应独立成行居中斜体排版，
其他未尽事宜，请参看“华东师范大学本科生毕业
论文格式要求”。

参考文献

- [1] Stewart Fraser 等. C# Xml 入门经典[M].北京：清华大学出版社,2003.
- [2] Morrison 等.Html 与 Xml 网页程序设计基础[M].北京：北京大学出版社,2002.
- [3] 思维科技、胡标. ASP 网络编程技术与实例[M].北京：人民邮电出版社,2005.
- [4] Nunit 官方网站[E]. <http://www.nunit.org>. 2005.
- [5] 微软 Commerce Server 产品概述[E].
<http://editblog.csdn.net/msdncolumn/archive/2005/02/25/1663.aspx>. 2005.
- [6] 陈建勋. XSLT 从入门到精通[M].北京：中国铁道出版社,2002.

理科专业要求有 6 篇以上的参考文献，按引用顺序排列。

中文宋体、五号
英文 Times New Roman、五号

参考文献具体格式如下：

（一）专著（注意应标明出版地及所参阅内容在原文献中的位置）

[序号] 作者.专著名[M].出版地：出版者，出版年，起止页.

（二）期刊中析出的文献（注意应标明年、卷、期，尤其注意区分卷和期）

[序号] 作者.题(篇)名[J].刊名.出版年，卷号(期号)：起止页.

（三）会议论文

[序号] 作者.篇名[C].会议名，会址，开会年.

（四）学位论文

[序号] 作者.题(篇)名[D].授学位地：授学位单位，授学位年.

（五）专利文献

[序号] 专利申请者.专利题名[P].专利国别，专利文献种类，专利号.出版日期.

（六）报纸文章

[序号] 作者.题(篇)名[N].报纸名.出版日期(版次).

（七）标准

[序号] 标准名称[S].出版地：出版年，起止页.

（八）报告

[序号] 作者.题(篇)名[R].报告年、月、日.

（九）电子文档

[序号] 作者.题(篇)名[E].出处或可获得地址(网址).发表或更新日期/引用日期(任选).

附录

1. 部分函数代码

- (1) LoadMessageFile 函数（过程），用于读取信息 XML 文件，取得相应节点的内容，并通过引用参数返回值。



中文宋体、五号

英文 Times New Roman、五号

致谢

不觉之间，已是别离时节。我于华师大逗留四载，未曾成就什么卓然大事，却也细细领会了点点滴滴的人情与暖意。校园间穿行，教室中学习，食堂里就餐，这些平凡的日常，串联起我的大学生活。

而今，论文终于告成，回顾一路走来，心怀感激与不舍，总归不能忘记那些助我前行的人们。首先，衷心感谢导师杨燕教授。在过去的几个月里，她耐心细致地为我解答疑惑，精心指导实验方案与论文撰写，遇难必解，费心费力。其次，感谢实习单位仙工智能所提供的良好氛围与办公环境。公司开放包容的企业文化与积极进取的团队精神，使我在实践中积累了宝贵的经验。我的毕业设计与论文也大半是在工位上写下的。此外，还需感谢陪伴我、鼓励我的朋友们。无论是学习中的相互扶持，还是生活中的倾心相伴，你们的存在让我的大学生活更加充实和美好。感激家人默默的信任、支持与理解。最后，也特别感谢劳神费力的诸位评审专家，感谢你们对我的论文认真负责的审阅和提出宝贵的意见。

五月日光和煦，清风吹拂。回望四年光阴，徒然感叹岁月飞逝，却也庆幸自己曾身处其中，感受温暖与成长。在此，谨向所有帮助过我的人表达诚挚的谢意。愿诸位安好，未来可期。